

Resumen Completo Programación III - 29/05/2026

Resumen Completo de Programación III (Clase del 29/05/2026)

1. Interfaces Modulares y Condicionales

- **Widgets Personalizados y Parámetros:** Creación de componentes modulares y reutilizables (como `CustomCard`). Permite definir constructores que reciban argumentos obligatorios u opcionales (`required`) para cambiar dinámicamente textos, URLs de imágenes o colores desde la pantalla principal.
- **Operador Ternario (`condicion ? verdadero : falso`):** Estructura condicional abreviada en una sola línea. Se utiliza directamente dentro del árbol de widgets para renderizar elementos de forma dinámica (por ejemplo, mostrar un contenedor vacío `Container()` si un parámetro es `null` o renderizar el widget completo si contiene datos).

2. Componentes Visuales y Carga Asíncrona

- **Tarjetas (`Card`):** Contenedores visuales con relieve. Propiedades clave: `elevation` para controlar la sombra/profundidad y `clipBehavior: Clip.antiAlias` para asegurar que el contenido interno (como imágenes) respete las esquinas redondeadas.
- **Imágenes Asíncronas (`FadeInImage`):** Carga optimizada de imágenes desde internet con `NetworkImage`. Muestra un `placeholder` (un GIF o imagen local ligera) mientras se descarga la imagen final, aplicando un efecto de desvanecimiento suave al completarse.

3. Estructura y Diseño Responsivo

- **Estructura de Pantallas:** Creación de vistas base con `Scaffold`, personalización del `AppBar` (`backgroundColor`) y centrado de elementos con `Center`.
- **Diseño Responsivo:** Uso de `width: double.infinity` para forzar a los contenedores a expandirse y adaptarse al ancho de cualquier dispositivo, evitando errores visuales de desbordamiento (`overflow`).
- **Decoración Avanzada:** Uso de `BoxDecoration` para suavizar esquinas (`borderRadius`), control de espacios exteriores (`margin`) y posicionamiento interno (`alignment`).

4. Interactividad y Animaciones

- **Gestión de Estado:** Cambio de `StatelessWidget` a `StatefulWidget` para manejar variables dinámicas de estado que actualizan la interfaz visual al cambiar de valor.
- **Interactividad:** Uso del `FloatingActionButton` como disparador de eventos de usuario y la clase `Random()` para generar valores visuales y alteraciones matemáticas aleatorias.
- **Animaciones Implícitas:** Reemplazo de `Container` por `AnimatedContainer` para realizar transiciones suaves y fluidas. Requiere uso obligatorio de `duration` y permite aplicar físicas naturales con curvas de aceleración (ej. `curve: Curves.decelerate`).